AI-Assisted Theorem Proving

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

AI-Assisted Theorem Proving
Computer-Assisted Theorem Proving (7CCSACTP)
Mohammad Abdulaziz

Where We Are in the Course

- **Covered in Weeks 1–4:**
  - **Week 1:** ITP fundamentals and the Isabelle/HOL environment.
  - **Week 2:** Mathematical induction and induction hypothesis generalisation.
  - **Week 3:** List recursion, structural induction, and proof automation (`simp`, `auto`, `nitpick`).
  - **Week 4:** Algebraic datatypes, termination size measures, and tree induction.

- In the first four weeks, we established the foundations of interactive theorem proving:
- Week 1: Introduction to ITPs and Isabelle/HOL basics.
- Week 2: Mathematical induction and induction hypothesis generalisation.
- Week 3: Functional programming on lists, structural list induction, and basic automation (`simp`, `auto`, `nitpick`).
- Week 4: Custom algebraic datatypes, primitive recursion, and tree induction.

This Week

- Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: **AI-assisted theorem proving**.

This Week
Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: **AI-assisted theorem proving**.

- Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: AI-assisted theorem proving.
- I will cover two types: classical machine-learning based approaches and modern machine learning-based approaches.

This Week

- Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: **AI-assisted theorem proving**.
- **Two types covered:**
  1. **Classical machine-learning based approaches:**
     - Premise selection & external automated provers (Sledgehammer).
  2. **Modern machine learning-based approaches:**
     - Reinforcement learning (RL) loops with compiler verifiers (AlphaProof, Aristotle).
     - Off-the-shelf coding agents via Model Context Protocol (MCP).
     - Two-stage Draft, Sketch, and Prove (DSP) architectures.

This Week
- Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: **AI-assisted theorem proving**.
- **Two types covered:**
  1. **Classical machine-learning based approaches:**
     - Premise selection & external automated provers (Sledgehammer).
  2. **Modern machine learning-based approaches:**
     - Reinforcement learning (RL) loops with compiler verifiers (AlphaProof, Aristotle).
     - Off-the-shelf coding agents via Model Context Protocol (MCP).
     - Two-stage Draft, Sketch, and Prove (DSP) architectures.

- Today I will introduce you to the last and most recent part of computer-assisted theorem provers' foundations: AI-assisted theorem proving.
- I will cover two types: classical machine-learning based approaches and modern machine learning-based approaches.

1. Sledgehammer

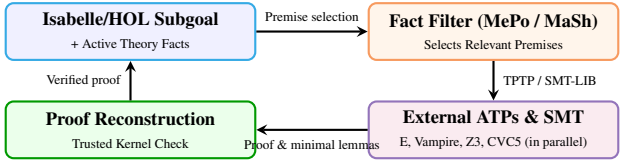
- **First & Most Impactful Automation Attempt:**
  - **"Sledgehammer"** in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
  - Bridges interactive theorem proving with external ATPs and SMT solvers.

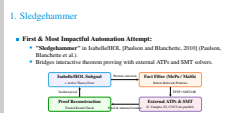
1. Sledgehammer
- **First & Most Impactful Automation Attempt:**
  - **"Sledgehammer"** in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
  - Bridges interactive theorem proving with external ATPs and SMT solvers.

- AI has been considered as a tool to improve theorem proving for a long time by now.
- The first and most impactful attempt was the development of Sledgehammer for Isabelle by Larry Paulson et al. [Paulson and Blanchette, 2010].
- Sledgehammer bridges interactive theorem proving with automated theorem provers (ATPs) and SMT solvers.
- Architecture: Fact Filter (MePo/MaSh) $\rightarrow$ TPTP/SMT-LIB $\rightarrow$ External Solvers (E, Vampire, Z3) $\rightarrow$ Proof Reconstruction in Isabelle Kernel.
- Live demonstration of Sledgehammer in Isabelle.

1. Sledgehammer

- **First & Most Impactful Automation Attempt:**
  - **"Sledgehammer"** in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
  - Bridges interactive theorem proving with external ATPs and SMT solvers.

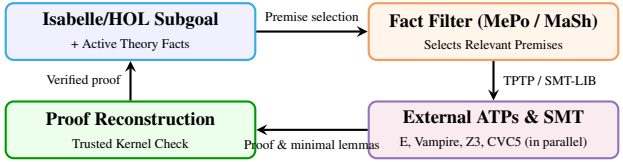




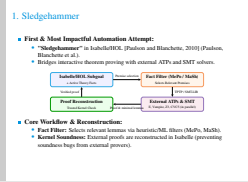
- AI has been considered as a tool to improve theorem proving for a long time by now.
- The first and most impactful attempt was the development of Sledgehammer for Isabelle by Larry Paulson et al. [Paulson and Blanchette, 2010].
- Sledgehammer bridges interactive theorem proving with automated theorem provers (ATPs) and SMT solvers.
- Architecture: Fact Filter (MePo/MaSh) → TPTP/SMT-LIB → External Solvers (E, Vampire, Z3) → Proof Reconstruction in Isabelle Kernel.
- Live demonstration of Sledgehammer in Isabelle.

1. Sledgehammer

- **First & Most Impactful Automation Attempt:**
  - **"Sledgehammer"** in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
  - Bridges interactive theorem proving with external ATPs and SMT solvers.



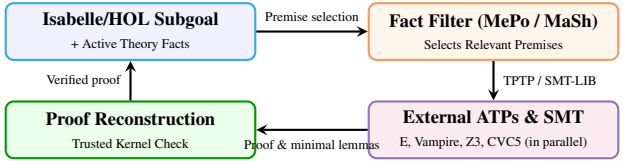
- **Core Workflow & Reconstruction:**
  - **Fact Filter:** Selects relevant lemmas via heuristic/ML filters (MePo, MaSh).
  - **Kernel Soundness:** External proofs are reconstructed in Isabelle (preventing soundness bugs from external provers).



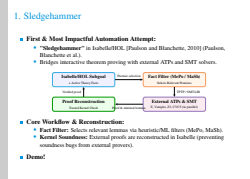
- AI has been considered as a tool to improve theorem proving for a long time by now.
- The first and most impactful attempt was the development of Sledgehammer for Isabelle by Larry Paulson et al. [Paulson and Blanchette, 2010].
- Sledgehammer bridges interactive theorem proving with automated theorem provers (ATPs) and SMT solvers.
- Architecture: Fact Filter (MePo/MaSh) → TPTP/SMT-LIB → External Solvers (E, Vampire, Z3) → Proof Reconstruction in Isabelle Kernel.
- Live demonstration of Sledgehammer in Isabelle.

1. Sledgehammer

- **First & Most Impactful Automation Attempt:**
  - **"Sledgehammer"** in Isabelle/HOL [Paulson and Blanchette, 2010] (Paulson, Blanchette et al.).
  - Bridges interactive theorem proving with external ATPs and SMT solvers.



- **Core Workflow & Reconstruction:**
  - **Fact Filter:** Selects relevant lemmas via heuristic/ML filters (MePo, MaSh).
  - **Kernel Soundness:** External proofs are reconstructed in Isabelle (preventing soundness bugs from external provers).
- **Demo!**



- AI has been considered as a tool to improve theorem proving for a long time by now.
- The first and most impactful attempt was the development of Sledgehammer for Isabelle by Larry Paulson et al. [Paulson and Blanchette, 2010].
- Sledgehammer bridges interactive theorem proving with automated theorem provers (ATPs) and SMT solvers.
- Architecture: Fact Filter (MePo/MaSh) → TPTP/SMT-LIB → External Solvers (E, Vampire, Z3) → Proof Reconstruction in Isabelle Kernel.
- Live demonstration of Sledgehammer in Isabelle.

2. Custom Models for Formal Proving

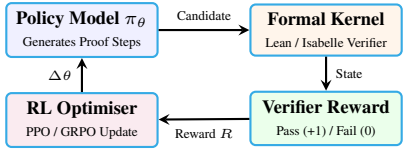
- **Key Players & Breakthroughs:**
  - **DeepMind** (AlphaProof), **Harmonic** (Aristotle), **Axiom Math** (AxiomProver).
  - **IMO Gold** achieved in 2025; **PutnamBench 2025** solved by $\geq$ 20 proving systems.

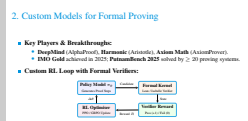
2. Custom Models for Formal Proving
Key Players & Breakthroughs:
DeepMind (AlphaProof), Harmonic (Aristotle), Axiom Math (AxiomProver).
IMO Gold achieved in 2025; PutnamBench 2025 solved by $\geq$ 20 proving systems.

- Proprietary systems initially dominated formal theorem proving by training custom models.
- Benchmarks like PutnamBench and the International Mathematical Olympiad (IMO) were considered mathematical holy grails.
- DeepMind developed AlphaProof by fine-tuning Gemini models with reinforcement learning (RL) search in Lean.
- Specialised startups like Harmonic (Aristotle) and Axiom Math (AxiomProver / AXLE) trained custom transformer policy and value models using RL feedback loops with theorem prover compilers.
- IMO Gold was achieved by AI in 2025 by DeepMind and Harmonic.
- PutnamBench 2025 is currently solved by at least 20 different proving systems.
- High compute and training cost: Base foundation pre-training costs \$5M–\$100M+, targeted RL stages cost \$250k–\$1M+ (e.g. DeepSeek-R1 $\approx$ \$294k), and test-time search costs \$60–\$300+ per solved theorem.

2. Custom Models for Formal Proving

- **Key Players & Breakthroughs:**
  - **DeepMind** (AlphaProof), **Harmonic** (Aristotle), **Axiom Math** (AxiomProver).
  - **IMO Gold** achieved in 2025; **PutnamBench 2025** solved by ≥ 20 proving systems.
- **Custom RL Loop with Formal Verifiers:**

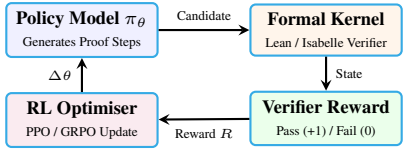




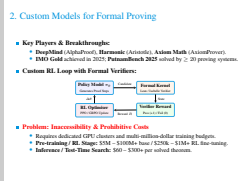
- Proprietary systems initially dominated formal theorem proving by training custom models.
- Benchmarks like PutnamBench and the International Mathematical Olympiad (IMO) were considered mathematical holy grails.
- DeepMind developed AlphaProof by fine-tuning Gemini models with reinforcement learning (RL) search in Lean.
- Specialised startups like Harmonic (Aristotle) and Axiom Math (AxiomProver / AXLE) trained custom transformer policy and value models using RL feedback loops with theorem prover compilers.
- IMO Gold was achieved by AI in 2025 by DeepMind and Harmonic.
- PutnamBench 2025 is currently solved by at least 20 different proving systems.
- High compute and training cost: Base foundation pre-training costs \$5M–\$100M+, targeted RL stages cost \$250k–\$1M+ (e.g. DeepSeek-R1 $\approx$ \$294k), and test-time search costs \$60–\$300+ per solved theorem.

2. Custom Models for Formal Proving

- **Key Players & Breakthroughs:**
  - **DeepMind** (AlphaProof), **Harmonic** (Aristotle), **Axiom Math** (AxiomProver).
  - **IMO Gold** achieved in 2025; **PutnamBench 2025** solved by $\geq$ 20 proving systems.
- **Custom RL Loop with Formal Verifiers:**



- **Problem: Inaccessibility & Prohibitive Costs**
  - Requires dedicated GPU clusters and multi-million-dollar training budgets.
  - **Pre-training / RL Stage:** \$5M – \$100M+ base / \$250k – \$1M+ RL fine-tuning.
  - **Inference / Test-Time Search:** \$60 – \$300+ per solved theorem.



- Proprietary systems initially dominated formal theorem proving by training custom models.
- Benchmarks like PutnamBench and the International Mathematical Olympiad (IMO) were considered mathematical holy grails.
- DeepMind developed AlphaProof by fine-tuning Gemini models with reinforcement learning (RL) search in Lean.
- Specialised startups like Harmonic (Aristotle) and Axiom Math (AxiomProver / AXLE) trained custom transformer policy and value models using RL feedback loops with theorem prover compilers.
- IMO Gold was achieved by AI in 2025 by DeepMind and Harmonic.
- PutnamBench 2025 is currently solved by at least 20 different proving systems.
- High compute and training cost: Base foundation pre-training costs \$5M–\$100M+, targeted RL stages cost \$250k–\$1M+ (e.g. DeepSeek-R1 $\approx$ \$294k), and test-time search costs \$60–\$300+ per solved theorem.

3. Democratisation via Agentic Proving

- **Off-the-Shelf Coding Agents (Batch Loop):**
  - Josef Urban et al. showed standard coding agents (e.g. *Claude Code*) formalised an entire textbook (*Munkres Topology*, 800+ theorems) using simple `isabelle build` for $\approx$ \$**200 / month**.

3. Democratisation via Agentic Proving
Off-the-Shelf Coding Agents (Batch Loop):
Josef Urban et al. showed standard coding agents (e.g. Claude Code) formalised an entire textbook (Munkres Topology, 800+ theorems) using simple isabelle build for ≈ $200 / month.

- Josef Urban and collaborators discovered that standard, off-the-shelf LLM coding agents are remarkably capable using simple batch builds (`isabelle build`).
- In a recent landmark experiment, standard coding agents formalised an entire 800+ theorem textbook on Topology (Munkres) for $\approx$ \$200/month.
- Following this, subsequent work connected off-the-shelf agents via Model Context Protocol (MCP) servers for direct, interactive proving.
- Any student or researcher with a standard coding agent and a laptop can now formalise mathematics and software without multi-million-dollar training clusters.

3. Democratisation via Agentic Proving

- **Off-the-Shelf Coding Agents (Batch Loop):**
  - Josef Urban et al. showed standard coding agents (e.g. *Claude Code*) formalised an entire textbook (*Munkres Topology*, 800+ theorems) using simple `isabelle build` for $\approx$ **$200 / month**.
- **Follow-up: Interactive Proving via MCP:**
  - Subsequent work connects agents directly via **Model Context Protocol (MCP)**.
  - Replaces batch CLI loops with rich, interactive kernel feedback (no cluster training needed).

3. Democratisation via Agentic Proving
- Off-the-Shelf Coding Agents (Batch Loop):
  - Josef Urban et al. showed standard coding agents (e.g. Claude Code) formalised an entire textbook (Munkres Topology, 800+ theorems) using simple isabelle build for ≈ $200 / month.
- Follow-up: Interactive Proving via MCP:
  - Subsequent work connects agents directly via Model Context Protocol (MCP).
  - Replaces batch CLI loops with rich, interactive kernel feedback (no cluster training needed).

- Josef Urban and collaborators discovered that standard, off-the-shelf LLM coding agents are remarkably capable using simple batch builds (`isabelle build`).
- In a recent landmark experiment, standard coding agents formalised an entire 800+ theorem textbook on Topology (Munkres) for $\approx$ $200/month.
- Following this, subsequent work connected off-the-shelf agents via Model Context Protocol (MCP) servers for direct, interactive proving.
- Any student or researcher with a standard coding agent and a laptop can now formalise mathematics and software without multi-million-dollar training clusters.

3. Democratisation via Agentic Proving

- **Off-the-Shelf Coding Agents (Batch Loop):**
  - Josef Urban et al. showed standard coding agents (e.g. *Claude Code*) formalised an entire textbook (*Munkres Topology*, 800+ theorems) using simple `isabelle build` for $\approx$ **$200 / month**.
- **Follow-up: Interactive Proving via MCP:**
  - Subsequent work connects agents directly via **Model Context Protocol (MCP)**.
  - Replaces batch CLI loops with rich, interactive kernel feedback (no cluster training needed).
- **Practical Advantages:**
  - No specialised GPU infrastructure or training required.
  - Fast, interactive feedback loop directly with the Isabelle/HOL environment.

3. Democratisation via Agentic Proving
- Off-the-Shelf Coding Agents (Batch Loop):
  - Josef Urban et al. showed standard coding agents (e.g. Claude Code) formalised an entire textbook (Munkres Topology, 800+ theorems) using simple isabelle build for ≈ $200 / month.
- Follow-up: Interactive Proving via MCP:
  - Subsequent work connects agents directly via Model Context Protocol (MCP).
  - Replaces batch CLI loops with rich, interactive kernel feedback (no cluster training needed).
- Practical Advantages:
  - No specialised GPU infrastructure or training required.
  - Fast, interactive feedback loop directly with the Isabelle/HOL environment.

- Josef Urban and collaborators discovered that standard, off-the-shelf LLM coding agents are remarkably capable using simple batch builds (`isabelle build`).
- In a recent landmark experiment, standard coding agents formalised an entire 800+ theorem textbook on Topology (Munkres) for $\approx$ $200/month.
- Following this, subsequent work connected off-the-shelf agents via Model Context Protocol (MCP) servers for direct, interactive proving.
- Any student or researcher with a standard coding agent and a laptop can now formalise mathematics and software without multi-million-dollar training clusters.

4. Technical Setup: Agentic LLMs for Theorem Proving

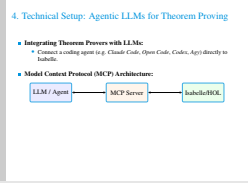
- **Integrating Theorem Provers with LLMs:**
  - Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.

4. Technical Setup: Agentic LLMs for Theorem Proving
Integrating Theorem Provers with LLMs:
Connect a coding agent (e.g. Claude Code, Open Code, Codex, Agy) directly to Isabelle.

- In this week I will demonstrate a new but strongly emerging way to apply theorem provers like Isabelle, via integration with LLMs.
- You should have a coding agent ready, e.g. Claude Code, Open Code, Codex, Agy, etc.
- In my demos I will use Claude Code.
- Main idea: Connect a coding agent to the theorem prover via a Model Context Protocol (MCP) server.
- An MCP server exposes Isabelle programmatically: write-file, process-file, save-file, etc.
- We will use PIDE-MCP (`https://github.com/kappelmann/isabelle-pide-mcp`) and AWS's Isabelle/Q (`https://github.com/awslabs/AutoCorrode`).

4. Technical Setup: Agentic LLMs for Theorem Proving

- **Integrating Theorem Provers with LLMs:**
  - Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.
- **Model Context Protocol (MCP) Architecture:**

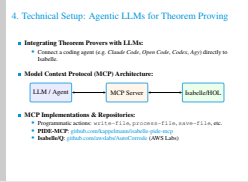


- In this week I will demonstrate a new but strongly emerging way to apply theorem provers like Isabelle, via integration with LLMs.
- You should have a coding agent ready, e.g. Claude Code, Open Code, Codex, Agy, etc.
- In my demos I will use Claude Code.
- Main idea: Connect a coding agent to the theorem prover via a Model Context Protocol (MCP) server.
- An MCP server exposes Isabelle programmatically: write-file, process-file, save-file, etc.
- We will use PIDE-MCP (`https://github.com/kappelmann/isabelle-pide-mcp`) and AWS's Isabelle/Q (`https://github.com/awslabs/AutoCorrode`).

4. Technical Setup: Agentic LLMs for Theorem Proving

- **Integrating Theorem Provers with LLMs:**
  - Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.
- **Model Context Protocol (MCP) Architecture:**

- **MCP Implementations & Repositories:**
  - Programmatic actions: `write-file`, `process-file`, `save-file`, etc.
  - **PIDE-MCP**: github.com/kappelmann/isabelle-pide-mcp
  - **Isabelle/Q**: github.com/awslabs/AutoCorrode (AWS Labs)

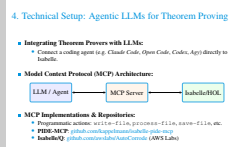


- In this week I will demonstrate a new but strongly emerging way to apply theorem provers like Isabelle, via integration with LLMs.
- You should have a coding agent ready, e.g. Claude Code, Open Code, Codex, Agy, etc.
- In my demos I will use Claude Code.
- Main idea: Connect a coding agent to the theorem prover via a Model Context Protocol (MCP) server.
- An MCP server exposes Isabelle programmatically: write-file, process-file, save-file, etc.
- We will use PIDE-MCP (`https://github.com/kappelmann/isabelle-pide-mcp`) and AWS's Isabelle/Q (`https://github.com/awslabs/AutoCorrode`).

4. Technical Setup: Agentic LLMs for Theorem Proving

- **Integrating Theorem Provers with LLMs:**
  - Connect a coding agent (e.g. *Claude Code*, *Open Code*, *Codex*, *Agy*) directly to Isabelle.
- **Model Context Protocol (MCP) Architecture:**

- **MCP Implementations & Repositories:**
  - Programmatic actions: `write-file`, `process-file`, `save-file`, etc.
  - **PIDE-MCP**: github.com/kappelmann/isabelle-pide-mcp
  - **Isabelle/Q**: github.com/awslabs/AutoCorrode (AWS Labs)
- **Demo!**



- In this week I will demonstrate a new but strongly emerging way to apply theorem provers like Isabelle, via integration with LLMs.
- You should have a coding agent ready, e.g. Claude Code, Open Code, Codex, Agy, etc.
- In my demos I will use Claude Code.
- Main idea: Connect a coding agent to the theorem prover via a Model Context Protocol (MCP) server.
- An MCP server exposes Isabelle programmatically: write-file, process-file, save-file, etc.
- We will use PIDE-MCP (`https://github.com/kappelmann/isabelle-pide-mcp`) and AWS's Isabelle/Q (`https://github.com/awslabs/AutoCorrode`).

5. Agentic Proving Architectures

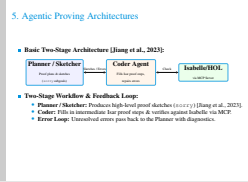
- **Basic Two-Stage Architecture [Jiang et al., 2023]:**

- The most basic agentic architecture for theorem proving consists of a Planner/Sketcher, a Coder, an MCP server, and Isabelle.
- Introduced in the "Draft, Sketch, and Prove" paradigm by Jiang, Li et al. [Jiang et al., 2023].
- The Planner/Sketcher produces detailed proof plans and sketches (e.g. sorried proofs).
- The Coder takes the proof plans & sketches, completes them, and ensures Isabelle accepts them via MCP.
- If an error cannot be resolved by the Coder, it passes back to the Planner/Sketcher.
- Since the area is moving very fast, I will leave a paper on KEATS for you to read and discuss in the LGT.

5. Agentic Proving Architectures

- **Basic Two-Stage Architecture [Jiang et al., 2023]:**

- **Two-Stage Workflow & Feedback Loop:**
  - **Planner / Sketcher:** Produces high-level proof sketches (`sorry`) [Jiang et al., 2023].
  - **Coder:** Fills in intermediate Isar proof steps & verifies against Isabelle via MCP.
  - **Error Loop:** Unresolved errors pass back to the Planner with diagnostics.

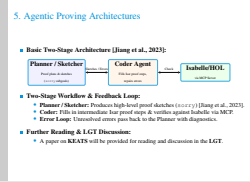


- The most basic agentic architecture for theorem proving consists of a Planner/Sketcher, a Coder, an MCP server, and Isabelle.
- Introduced in the "Draft, Sketch, and Prove" paradigm by Jiang, Li et al. [Jiang et al., 2023].
- The Planner/Sketcher produces detailed proof plans and sketches (e.g. sorried proofs).
- The Coder takes the proof plans & sketches, completes them, and ensures Isabelle accepts them via MCP.
- If an error cannot be resolved by the Coder, it passes back to the Planner/Sketcher.
- Since the area is moving very fast, I will leave a paper on KEATS for you to read and discuss in the LGT.

5. Agentic Proving Architectures

- **Basic Two-Stage Architecture [Jiang et al., 2023]:**

- **Two-Stage Workflow & Feedback Loop:**
  - **Planner / Sketcher:** Produces high-level proof sketches (`sorry`) [Jiang et al., 2023].
  - **Coder:** Fills in intermediate Isar proof steps & verifies against Isabelle via MCP.
  - **Error Loop:** Unresolved errors pass back to the Planner with diagnostics.
- **Further Reading & LGT Discussion:**
  - A paper on **KEATS** will be provided for reading and discussion in the **LGT**.



- The most basic agentic architecture for theorem proving consists of a Planner/Sketcher, a Coder, an MCP server, and Isabelle.
- Introduced in the "Draft, Sketch, and Prove" paradigm by Jiang, Li et al. [Jiang et al., 2023].
- The Planner/Sketcher produces detailed proof plans and sketches (e.g. sorried proofs).
- The Coder takes the proof plans & sketches, completes them, and ensures Isabelle accepts them via MCP.
- If an error cannot be resolved by the Coder, it passes back to the Planner/Sketcher.
- Since the area is moving very fast, I will leave a paper on KEATS for you to read and discuss in the LGT.

6. Current State & Industrial Adoption

- **Specialised Startups:**
  - Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.

6. Current State & Industrial Adoption
- **Specialised Startups:**
  - Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.

- AI-assisted proving and autoformalization are directly solving open mathematical problems.
- Specialised companies have secured significant venture capital funding (e.g. Harmonic, Math Inc, Axiom Math).
- Industry formal verification:
  - Apple corecrypto: `security.apple.com`
  - AWS Nitro: `amazon.science/nitro-hypervisor`
- Mathematical breakthroughs:
  - Terence Tao: `terrytao.wordpress.com`
  - OpenAI Astra: `github.com/openai/ten-proofs`
  - Claude: `oeis.org/A000729`
  - Axiom Math: `primegaps.axiommath.ai`

6. Current State & Industrial Adoption

- **Specialised Startups:**
  - Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.
- **Formal Verification in Industry:**
  - **Apple**: Post-quantum cryptography verification in *corecrypto* via AutoCorres2.
  - **AWS**: Automated verification for hypervisors (Nitro Hypervisor) and cloud security policies.
  - **Arm & Intel**: Hardware, firmware, and instruction set architecture verification.

6. Current State & Industrial Adoption
- Specialised Startups:
  - Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.
- Formal Verification in Industry:
  - Apple: Post-quantum cryptography verification in *corecrypto* via AutoCorres2.
  - AWS: Automated verification for hypervisors (Nitro Hypervisor) and cloud security policies.
  - Arm & Intel: Hardware, firmware, and instruction set architecture verification.

- AI-assisted proving and autoformalization are directly solving open mathematical problems.
- Specialised companies have secured significant venture capital funding (e.g. Harmonic, Math Inc, Axiom Math).
- Industry formal verification:
  - Apple corecrypto: `security.apple.com`
  - AWS Nitro: `amazon.science/nitro-hypervisor`
- Mathematical breakthroughs:
  - Terence Tao: `terrytao.wordpress.com`
  - OpenAI Astra: `github.com/openai/ten-proofs`
  - Claude: `oeis.org/A000729`
  - Axiom Math: `primegaps.axiommath.ai`

6. Current State & Industrial Adoption

- **Specialised Startups:**
  - Companies developing formal reasoning tools: *Harmonic*, *Math Inc*, *Axiom Math*.
- **Formal Verification in Industry:**
  - **Apple**: Post-quantum cryptography verification in *corecrypto* via AutoCorres2.
  - **AWS**: Automated verification for hypervisors (Nitro Hypervisor) and cloud security policies.
  - **Arm & Intel**: Hardware, firmware, and instruction set architecture verification.
- **Mathematical Research & Breakthroughs:**
  - **Terence Tao**: Integrating autoformalization and agentic provers into active research.
  - **OpenAI (Astra)**: Solved 10 open problems with machine-checked Lean certificates.
  - **Claude**: Discovered Hadamard matrices for all open orders < 2000 (e.g. order 668).
  - **Axiom Math**: Proved prime gap bound ≤ 212 (reduced from 246) in Lean 4.

- AI-assisted proving and autoformalization are directly solving open mathematical problems.
- Specialised companies have secured significant venture capital funding (e.g. Harmonic, Math Inc, Axiom Math).
- Industry formal verification:
  - Apple corecrypto: `security.apple.com`
  - AWS Nitro: `amazon.science/nitro-hypervisor`
- Mathematical breakthroughs:
  - Terence Tao: `terrytao.wordpress.com`
  - OpenAI Astra: `github.com/openai/ten-proofs`
  - Claude: `oeis.org/A000729`
  - Axiom Math: `primegaps.axiommath.ai`

7. Closing Remarks

- **Core Skills You Must Master Yourself:**
  1. Splitting problems into manageable formal subproblems.
  2. Formally modelling problem domains.
  3. Formulating mathematically sound, correct specifications.
  4. Constructing correct, readable, concise, and fast-checking proofs (Isabelle linter).

7. Closing Remarks
Core Skills You Must Master Yourself:
1. Splitting problems into manageable formal subproblems.
2. Formally modelling problem domains.
3. Formulating mathematically sound, correct specifications.
4. Constructing correct, readable, concise, and fast-checking proofs (Isabelle linter).

- Even with modern AI assistance, you still need to master the foundational formalisation skills yourself:
- (1) Splitting complex problems into manageable formal subproblems.
- (2) Formally modelling problem domains.
- (3) Formulating mathematically sound, correct specifications.
- (4) Constructing correct, readable, concise, and fast-checking proofs.

References I

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs, February 2023.

Lawrence C Paulson and Jasmin Christian Blanchette. Three years of experience with sledgehammer, a practical link between automatic and interactive theorem provers. In *PAAR@ IJCAR*, pages 1–10, 2010.

References I

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs, February 2023.

Lawrence C Paulson and Jasmin Christian Blanchette. Three years of experience with sledgehammer, a practical link between automatic and interactive theorem provers. In *PAAR@ IJCAR*, pages 1–10, 2010.